

LABORATORY OBSERVATION

Course: Full Stack Web Development Course

Code: 23CM4121

Experiment No.: Observation Task-05

Student Name: Ganji Venkata Vijay Anudeep

Roll No.: A24126552081

Date: 25-09-2026

1. TITLE

Comprehensive Implementation of Client-Side Interactivity, Node.js File Management, Express.js Routing, and DOM Manipulation.

2. AIM

To develop and demonstrate full-stack web development concepts by building an interactive product calculator, a server-side file management script, a RESTful Express.js student directory, and a dynamic DOM manipulation interface.

3. OBJECTIVE

The objectives of these implementations are:

- To combine JavaScript functions, variables, events, and DOM updates to create interactive web applications.
- To work with the Node.js File System (fs) module, perform file operations, and develop basic server-side scripts.
- To configure an Express.js server, implement routing, HTTP methods, and response handling for a REST-style API.
- To understand the DOM structure and manipulate HTML elements, inline styles, and attributes dynamically using JavaScript.

4. THEORY

Modern web applications require a blend of client-side and server-side technologies. HTML and CSS form the foundational structure and styling. Client-side JavaScript provides real-time interactivity, utilizing the Document Object Model (DOM) to capture user events (like input or click) and dynamically update the interface without page reloads. On the server side, Node.js offers a runtime environment allowing JavaScript to interact directly with the operating system, such as managing local files via the fs and readline modules. Express.js acts as a lightweight framework built on top of Node.js, simplifying the creation of web servers, routing HTTP requests (like GET), and sending structured responses (like JSON or HTML payloads) back to the client.

5. ALGORITHM/PROCEDURE

1. **Environment Setup:** Initialize the required HTML structures for browser-based tasks and set up a

Node.js environment (with the Express package installed) for server-based tasks.

2. **Client-Side Interactivity:** Select DOM elements using `getElementById`. Attach event listeners to input fields and buttons to trigger JavaScript functions that perform real-time calculations or DOM modifications.
3. **Server-Side File Management:** Import the `fs` and `readline` modules. Prompt the user for input via the terminal, execute synchronous file writing (`writeFileSync`), appending (`appendFileSync`), and reading (`readFileSync`), then log the outputs sequentially.
4. **API Routing:** Instantiate an Express application, define routes (`/`, `/students`, `/about`) using `app.get()`, and serve corresponding JSON arrays or HTML content. Start the server on a specified port.
5. **Execution & Validation:** Execute web pages in a modern browser and Node.js scripts in the terminal. Verify the required outputs dynamically and through tools like Postman or the browser for API endpoints.

6. PROGRAM

6.1 Interactive Product Calculator

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Interactive Product Calculator</title>
</head>
<body>
  <h2>Product Calculator</h2>

  <!-- Input fields for product details -->
  <label>Product Name: <input type="text" id="pName" value="Headphones"></label><br><br>
  <label>Quantity: <input type="number" id="qty" value="1" min="1"></label><br><br>
  <label>Price ($): <input type="number" id="price" value="50" min="0"></label><br><br>

  <!-- Dynamic output display -->
  <h3>Total Price: $<span id="totalPrice">50.00</span></h3>

  <script>
    // Select DOM elements
    const qtyInput = document.getElementById('qty');
    const priceInput = document.getElementById('price');
    const totalDisplay = document.getElementById('totalPrice');

    // Function to calculate and update the total price
    function calculateTotal() {
```

```

    // Parse values, defaulting to 0 if invalid
    const quantity = parseFloat(qtyInput.value) || 0;
    const price = parseFloat(priceInput.value) || 0;

    // Update the DOM with the formatted total
    totalDisplay.textContent = (quantity * price).toFixed(2);
  }

  // Event listeners trigger recalculation on any input change
  qtyInput.addEventListener('input', calculateTotal);
  priceInput.addEventListener('input', calculateTotal);
</script>
</body>
</html>

```

6.2 Node.js File Management

fileManager.js

```

const fs = require('fs');
const readline = require('readline');

// Setup readline interface to accept user input from the terminal
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

rl.question('Enter the filename (e.g., test.txt): ', (filename) => {
  rl.question('Enter the initial file content: ', (content) => {

    // 1. Create and write to the file
    fs.writeFileSync(filename, content);
    console.log(`\n[Success] File '${filename}' created and written.`);

    // 2. Read and display the current contents
    const initialRead = fs.readFileSync(filename, 'utf8');
    console.log(`[Read] Current contents: ${initialRead}`);

    // 3. Append additional content to the file
    const additionalContent = '\nAppended Data: This line was added subsequently.';
    fs.appendFileSync(filename, additionalContent);
    console.log(`[Success] Additional content appended.`);
  });
});

```

```

// 4. Display the final file contents
const finalRead = fs.readFileSync(filename, 'utf8');
console.log(`\n--- Final Contents of ${filename} ---\n${finalRead}`);

// Close the input stream
rl.close();
});
});

```

6.3 Express.js Student Server

server.js

```

const express = require('express');
const app = express();
const PORT = 3000;

// Route 1: Root endpoint '/'
app.get('/', (req, res) => {
  res.send('<h2>Welcome to the Student Directory API</h2><p>Visit /students or /about.</p>');
});

// Route 2: '/students' returning a list of at least five students
app.get('/students', (req, res) => {
  const students = [
    { id: 1, name: 'Alice Smith', major: 'Computer Science' },
    { id: 2, name: 'Bob Johnson', major: 'Artificial Intelligence' },
    { id: 3, name: 'Charlie Brown', major: 'Machine Learning' },
    { id: 4, name: 'Diana Prince', major: 'Data Science' },
    { id: 5, name: 'Evan Wright', major: 'Software Engineering' }
  ];
  // Returns the student array as JSON
  res.json(students);
});

// Route 3: '/about' displaying information about the application
app.get('/about', (req, res) => {
  res.send('<h2>About This App</h2><p>This Express.js application serves basic student records.</p>');
});

// Initialize the server

```

```
app.listen(PORT, () => {
  console.log(`Server is running and listening on http://localhost:${PORT}`);
});
```

6.4 DOM Explanation & Demonstration

domDemo.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>DOM Manipulation Demo</title>
  <style>
    /* Initial base styling */
    .box {
      padding: 30px;
      border: 2px solid #333;
      width: 250px;
      text-align: center;
      font-family: sans-serif;
      margin-bottom: 15px;
    }
  </style>
</head>
<body>

  <!-- Target Element -->
  <div id="targetElement" class="box" data-status="original" title="Unmodified Element">
    Original Content
  </div>

  <!-- Trigger Button -->
  <button onclick="modifyElement()">Manipulate DOM</button>

  <script>
    function modifyElement() {
      // Target the specific element in the DOM tree
      const el = document.getElementById('targetElement');

      // 1. Change Content
      el.innerHTML = "<strong>Content Dynamically Changed!</strong>";
```

```

// 2. Change Style
el.style.backgroundColor = "#e0f7fa";
el.style.color = "#006064";
el.style.borderRadius = "12px";
el.style.borderColor = "#00acc1";

// 3. Change Attributes
el.setAttribute('data-status', 'modified'); // Update custom data attribute
el.title = "Element successfully modified by JavaScript"; // Update title attribute
}
</script>
</body>
</html>

```

7. CODE EXPLANATION

7.1 Interactive Product Calculator

Code / Function	Explanation
<code>parseFloat()</code>	Converts the user input string from the HTML fields into a floating-point number to allow mathematical operations.
<code>.toFixed(2)</code>	Formats the calculated total price to always display exactly two decimal places (e.g., 50.00).
<code>addEventListener('input', ...)</code>	Attaches an event listener that fires immediately upon any change to the input fields, enabling real-time calculation without refreshing.

7.2 Node.js File Management

Code / Function	Explanation
<code>readline.createInterface()</code>	Initializes a terminal interface that allows the Node.js script to prompt for and read user input from the standard input stream.

<code>fs.writeFileSync()</code>	A synchronous method that creates a new file (or overwrites an existing one) and writes the specified content into it.
<code>fs.appendFileSync()</code>	A synchronous method that adds new string content to the end of an existing file without overwriting the original data.

7.3 Express.js Student Server

Code / Function	Explanation
<code>express()</code>	Initializes the Express application instance, granting access to routing and middleware methods.
<code>app.get()</code>	Defines a routing handler for HTTP GET requests matching the specified URL path (e.g., <code>/students</code>).
<code>res.json()</code>	Automatically formats the provided JavaScript object or array into a JSON string and sets the appropriate headers before sending the response to the client.

7.4 DOM Explanation & Demonstration

Code / Function	Explanation
<code>element.innerHTML</code>	Allows the script to overwrite the HTML content nested inside the selected DOM element, rendering any HTML tags provided in the string.
<code>element.style.propertyName</code>	Directly modifies the inline CSS style of the targeted element dynamically (e.g., changing background color).

element.setAttribute()	Updates or creates an HTML attribute on the selected element (e.g., changing a data-status attribute).
------------------------	--

8. TEST CASES AND EXECUTION DATA OUTPUT

8.1 Interactive Product Calculator

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Open page, defaults loaded	Total shows \$50.00	Total shows \$50.00	Pass
TC-02	Change Quantity to 3	Total dynamically updates to \$150.00	Total updates instantly to \$150.00	Pass
TC-03	Clear Price field entirely	Code falls back to 0, Total shows \$0.00	Total shows \$0.00	Pass

Product Calculator

Product Name:

Quantity:

Price (\$):

Total Price: \$50.00

Product Calculator

Product Name:

Quantity:

Price (\$):

Total Price: \$200.00

8.2 Node.js File Management

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Filename: data.txt, Content: Hello Node	File is created; script logs success message and current content	Logs [Success] and prints Hello Node	Pass
TC-02	Append operation execution	Appends text; final read displays initial + appended string on a new line	Final string output displays both parts cleanly	Pass

```
● ~/full-stack-web-development/01/task-05/src | main !11 ?4 node node-file-management.js
Enter the filename (e.g., test.txt): test.txt
Enter the initial file content: hello

[Success] File 'test.txt' created and written.
[Read] Current contents: hello
[Success] Additional content appended.

--- Final Contents of test.txt ---
hello
Appended Data: This line was added subsequently.

○ ~/full-stack-web-development/01/task-05/src | main !11 ?4
```

```
01-observation > task-05-pbl_part1 > src > test.txt
1  hello
2  Appended Data: This line was added subsequently.
```

8.3 Express.js Student Server

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Navigate browser to http://localhost:3000/	Renders HTML Welcome...	Browser displays the correct welcome HTML	Pass
TC-02	Navigate browser to /students	Returns raw JSON array containing 5 student objects	Browser displays structured JSON array	Pass
TC-03	Navigate browser to /invalid-path	Express handles as 404 Not Found automatically	Browser shows Cannot GET /invalid-path	Pass

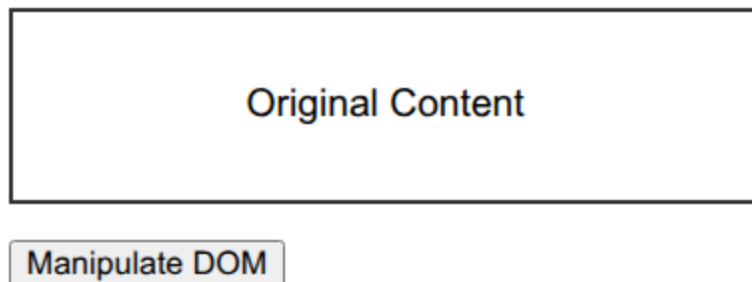
```
● ~/full-stack-web-development/01/task-05/src | main !11 ?4 node express-student-server.js
Server is running and listening on http://localhost:3000

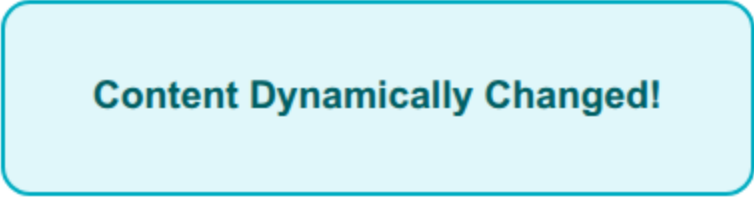
○ ~/full-stack-web-development/01/task-05/src | main !11 ?4
```

8.4 DOM Explanation & Demonstration

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Hover over target div before click	Tooltip shows Unmodified Element (from title attribute)	Tooltip displays default text	Pass
TC-02	Click "Manipulate DOM" button	Text turns bold; box changes color to light blue with rounded corners	Visual styling and text instantly update	Pass
TC-03	Hover over div after click	Tooltip now shows updated title text	Tooltip shows updated JS text	Pass

Output:





Content Dynamically Changed!



Manipulate DOM

9. RESULT

Thus, the four Project-Based Learning objectives were successfully implemented. The Interactive Calculator dynamically handled DOM events, the Node.js script successfully managed file I/O operations via the terminal, the Express.js server correctly routed and responded to varied HTTP requests, and the DOM manipulation interface efficiently altered element properties inline. All behaviors were verified successfully through structural test cases.